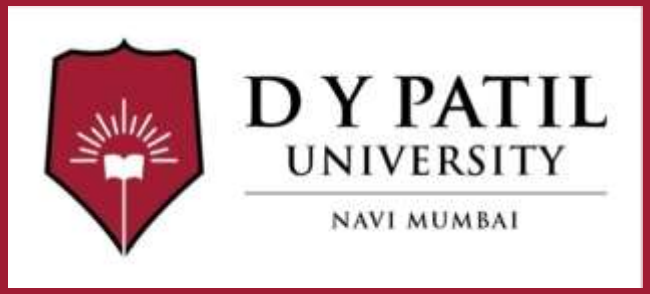
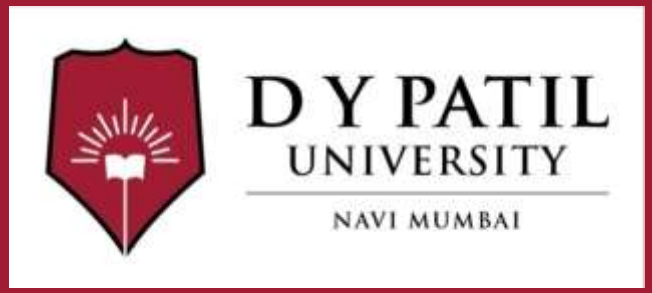


Subject Name :Web Development with Java

Unit No:I

Unit Name :Java Database Connectivity

Faculty Name :Dr. Saloni Bhushan



Unit No.1

Faculty Name :

Index

Unit No:1

Lecture No: 1

Content

Java Database Connectivity

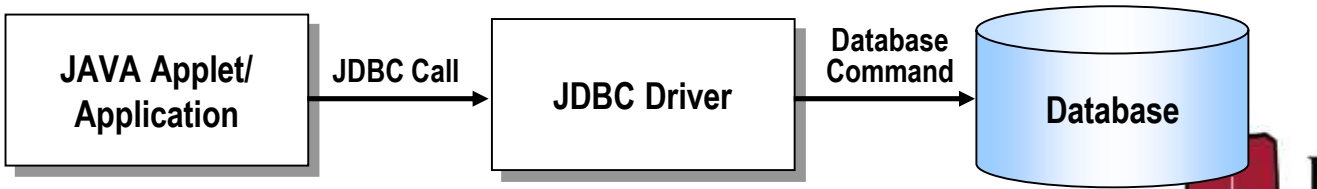
- Introduction to JDBC and its role in Java applications
- JDBC architecture and components
- Types of JDBC drivers
- Steps to create a JDBC application
- Types of JDBC statements:
  - Statement
  - PreparedStatement
  - CallableStatement
- Types of ResultSet:
  - Scrollable ResultSet
  - Updatable ResultSet
- Database operations using JDBC:
  - Insert, Update, Select, Delete
- Development of console-based JDBC applications

Overview (1/2)

- JDBC
  - **JDBC (Java Database Connectivity)** is a standard **Java API** that enables Java applications to interact with databases.
  - It provides a set of **classes and interfaces** to connect to a database, execute SQL queries, and process the results. Classes and Interfaces are in the ***java.sql*** package
  - JDBC acts as a **bridge between Java applications and databases** such as MySQL, Oracle, PostgreSQL, SQL Server, etc.

- Need of JDBC

  Most real-world applications need to **store, retrieve, update, and delete data**.
  Java alone cannot directly communicate with databases, so JDBC is required to:
  - Establish a connection with a database
  - Send SQL commands from Java to the database
  - Receive and process results in Java form
  - Ensure database-independent programming



Overview (2/2)

- Reason for JDBC
  - Database vendors (Microsoft Access, MySQL, Oracle etc.) provide proprietary (non standard) API for **sending** SQL to the server and **receiving** results from it
  - Languages such as C/C++ can make use of these proprietary APIs directly
    - High performance
    - Can make use of non standard features of the database
    - All the database code needs to be rewritten if you change database vendor or product
  - JDBC is a **vendor independent** API for accessing relational data from different database vendors in a consistent way

D Y PATIL UNIVERSITY
NAVI MUMBAI

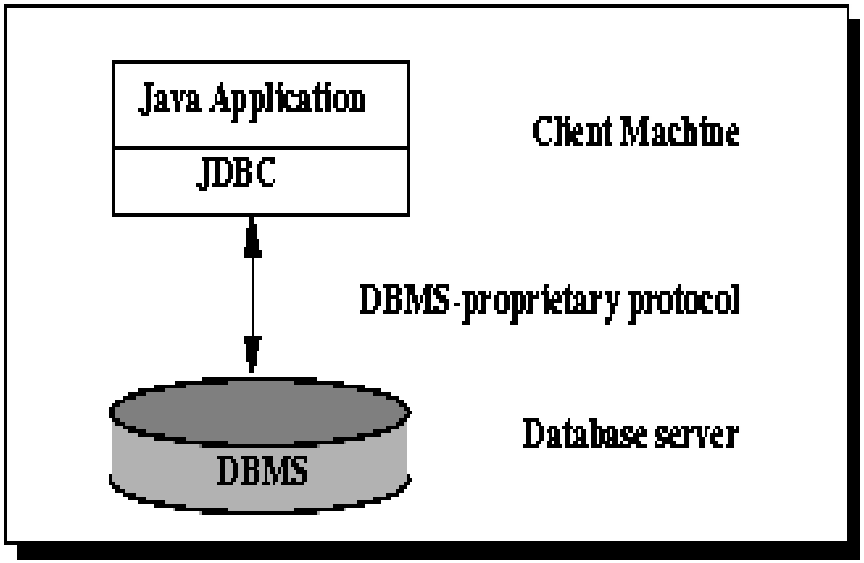
JDBC Architecture

- JDBC architecture can be implemented using **two-tier** or **three-tier architecture**, depending on how the application is designed.

- Two-tier and Three-tier Processing Models
  - The JDBC API supports both two-tier and three-tier processing models for database access.

Two-tier Architecture for Data Access.

- In **two-tier architecture**, the **Java application directly communicates with the database** using JDBC.



The client application contains:

- User Interface
- Business logic
- JDBC code

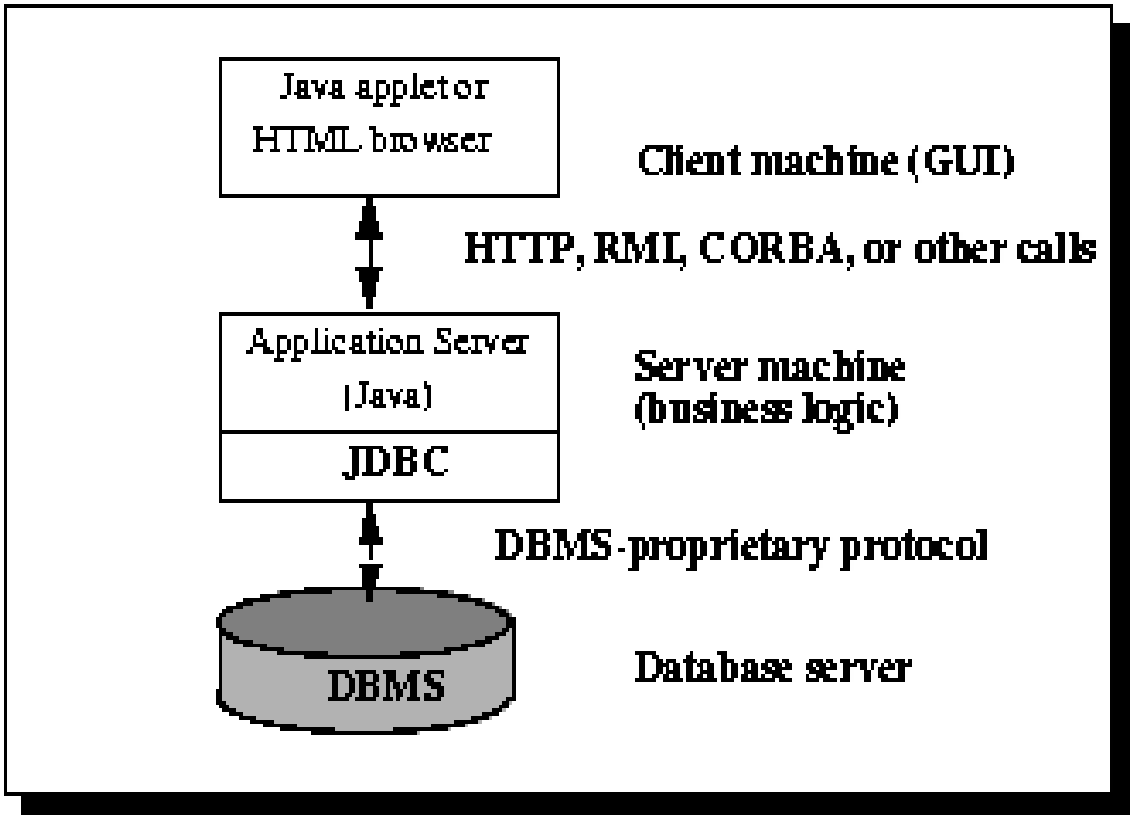
- JDBC driver directly connects the client to the database.
- SQL queries are executed directly from the client.

D Y PATIL UNIVERSITY
NAVI MUMBAI

- Example Applications
  - Java Swing applications
  - Standalone desktop applications
  - Small-scale systems
- Advantages
  - Simple to design and implement
  - Faster communication (direct connection)
  - Suitable for small applications
- Disadvantages
  - Poor scalability
  - Database credentials exposed to client
  - Difficult to manage for large users

Three-tier Architecture for Data Access.

In **three-tier architecture**, the client does **not directly access the database**.
A **middle layer (application server)** handles JDBC operations.



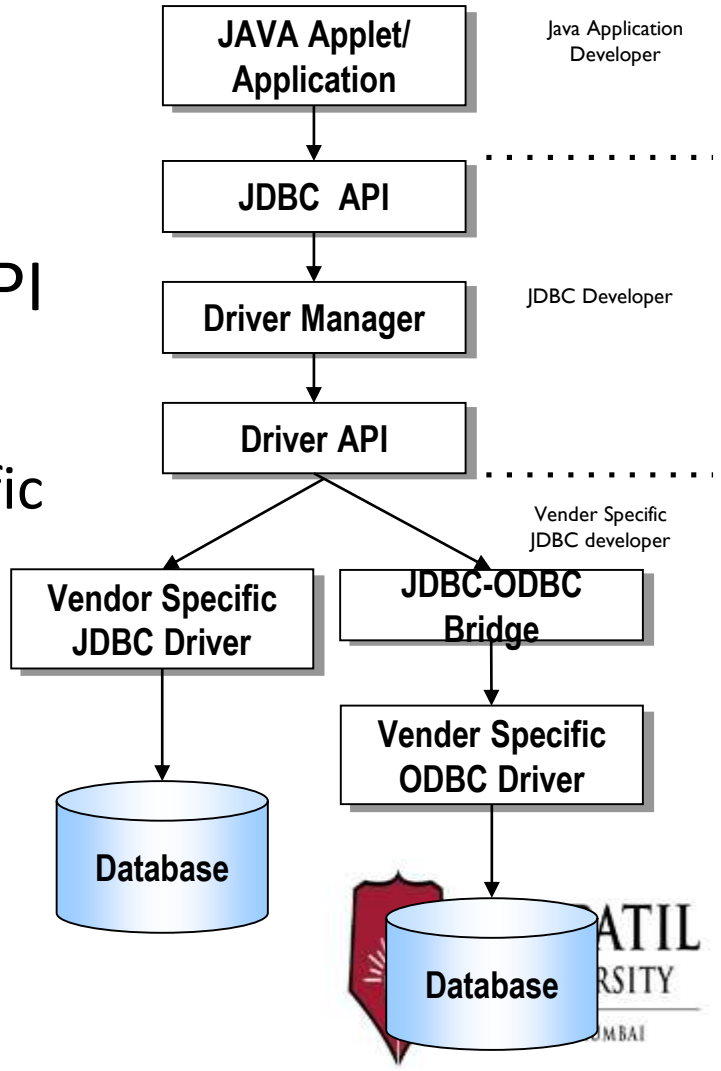
D Y PATIL
UNIVERSITY
NAVI MUMBAI

- **Explanation**
- Client layer:
  - User interface (browser, desktop app)
- Middle layer:
  - Business logic
  - JDBC code
  - Security and transaction management
- Database layer:
  - Stores and manages data
- **Example Applications**
- Web applications using JSP/Servlets
- Enterprise applications
- Banking and e-commerce systems

- **Advantages**
- High scalability
- Better security
- Easy maintenance
- Supports large number of users
- **Disadvantages**
- More complex design
- Higher development cost
- Requires application server

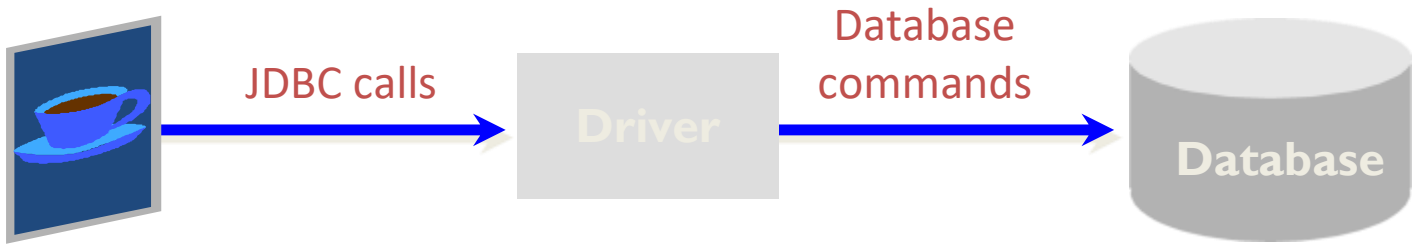
JDBC Model

- JDBC consists of two parts:
  - JDBC API, a purely Java-based API
  - JDBC driver manager
    - Communicates with vendor-specific drivers



A JDBC Driver

- Is an interpreter that translates JDBC method calls to vendor-specific database commands



- Implements interfaces in java.sql
- Can also provide a vendor's extensions to the JDBC standard

JDBC Driver Type

- JDBC-ODBC bridge plus ODBC driver
- Native-API partly-Java driver
- Network Protocol Driver - Type 3 Driver (Fully Java Driver)
- Thin Driver - Type 4 Driver (Fully Java Driver)

JDBC-ODBC bridge plus ODBC driver

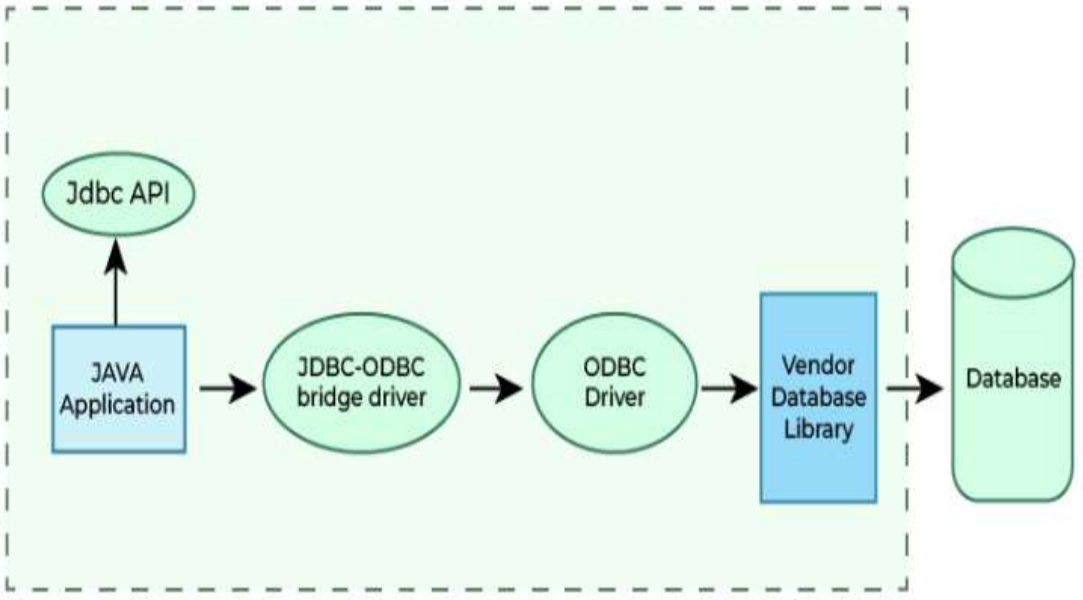
Converts JDBC calls into ODBC calls

Platform dependent

Very slow performance

Removed after Java 8

Usage: Learning purpose only



Advantages

- This driver software is built-in with JDK so no need to install separately.
- It is a database independent driver.

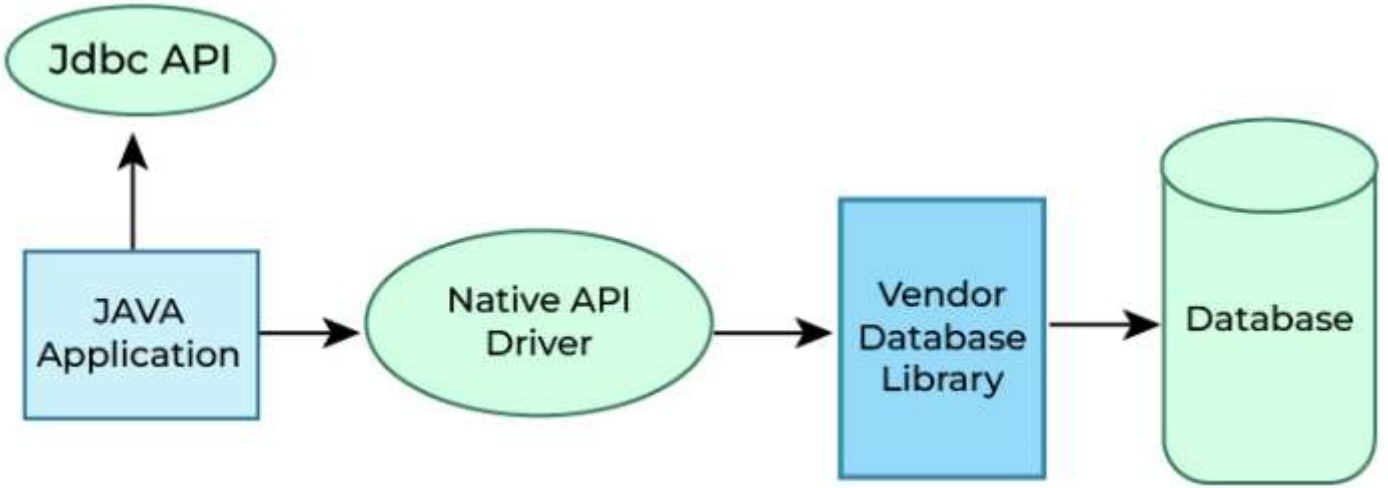
Disadvantages

- The data transferred through this driver is not so secured.
- The ODBC bridge driver is needed to be installed in individual client machines.
- Type-1 driver isn't written in java, that's why it isn't a portable driver.

2.

2. Native-API Driver - Type 2 Driver (Partially Java Driver)

Uses database-specific native libraries
Faster than Type 1
Platform dependent
Requires the client -side libraries of the database
not fully written in Java that is why it is also called Partially Java driver



D Y PATIL UNIVERSITY
NAVI MUMBAI

2. Native-API Driver - Type 2 Driver (Partially Java Driver)

Advantage

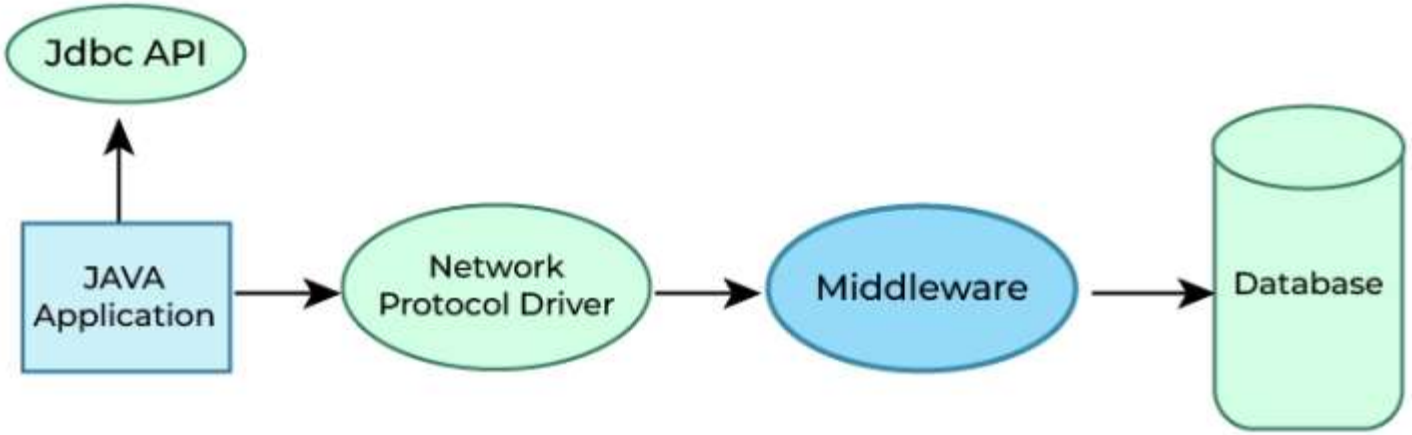
- Native-API driver gives better performance than JDBC-ODBC bridge driver.
- More secure compared to the type-1 driver.

Disadvantages

- Driver needs to be installed separately in individual client machines
- The Vendor client library needs to be installed on client machine.
- Type-2 driver isn't written in java, that's why it isn't a portable driver
- It is a database dependent driver.

3. Network Protocol Driver - Type 3 Driver (Fully Java Driver)

- Uses middleware between client and database
- Platform independent
- No client-side database libraries required
- Slower due to network overhead



Advantages

- Type-3 drivers are fully written in Java, hence they are portable drivers.
- No client side library is required
- Database independent , Easy to switch databases

Disadvantages

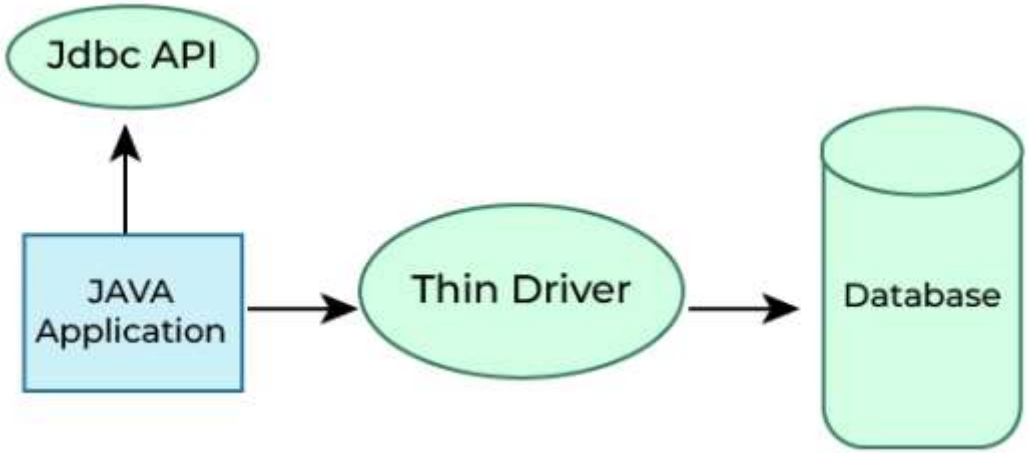
- Network support is required on client machine.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

4. Thin Driver - Type 4 Driver (Fully Java Driver)

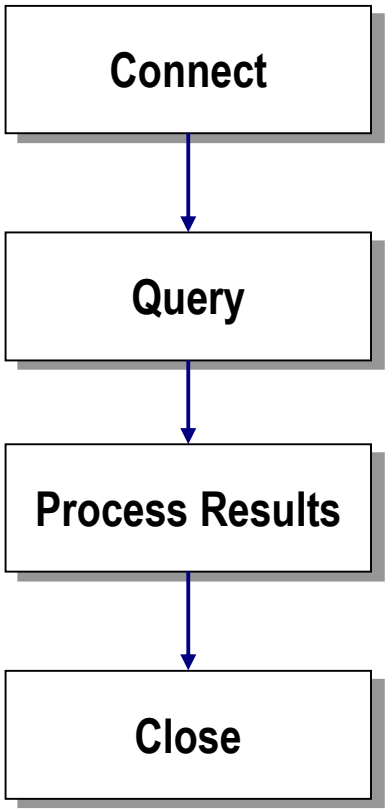
- Pure Java driver
- Platform independent
- Best performance
- Most widely used driver

Example:

MySQL → com.mysql.cj.jdbc.Driver



JDBC Programming Steps



1) Register the driver
2) Create a connection to the database

1) Create a statement
2) Query the database

1) Get a result set
2) Assign results to Java variables

1) Close the result set
2) Close the statement
3) Close the connection

Difference between type 3 & type 4

Type 3 Driver uses a middleware server to communicate with the database.

JDBC calls are sent over the network to the middleware, which converts them into database-specific calls.

It is platform independent but slower due to network overhead.

Type 4 Driver directly converts JDBC calls into database protocol calls.

It does not require middleware or native libraries.

It is platform independent and provides the best performance.

Therefore, Type 4 driver is most commonly used.

Skeleton Code

```
Class.forName(DRIVERNAME);

Connection con = DriverManager.getConnection(
                    CONNECTIONURL, DBID, DBPASSWORD);

Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT a, b, c FROM member);

While(rs.next())
{
      Int x = rs.getInt("a");
      String s = rs.getString("b");
      Float f = rs.getFloat("c");
}

rs.close();
stmt.close();
con.close();
```

Loading a JDBC driver

Connecting to a database

Executing SQL

Processing the result set

Closing the connections

Step 1 : Loading a JDBC Driver

- A JDBC driver is needed to connect to a database
- Loading a driver requires the class name of the driver.
  Ex

com.mysql.cj.jdbc.Driver

- Loading the driver class

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

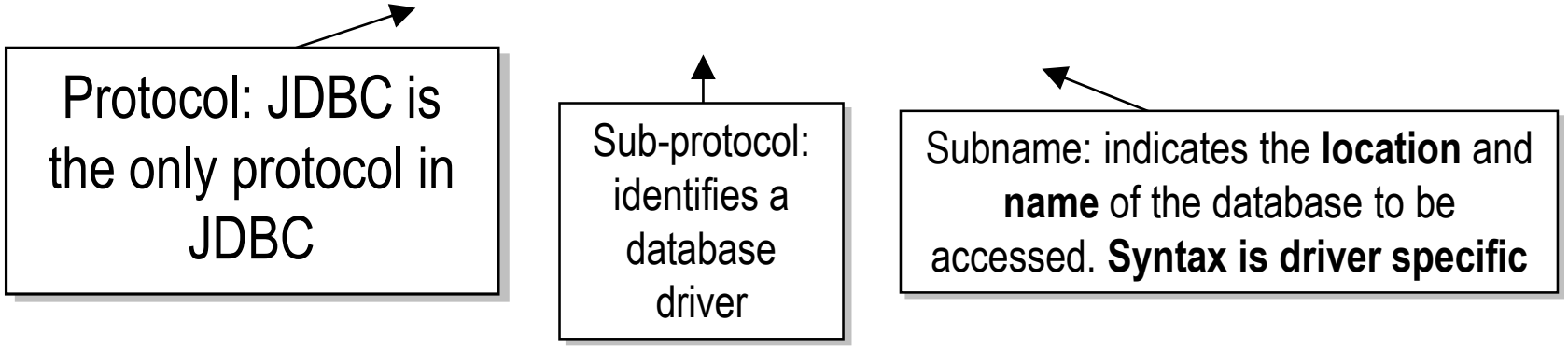
- It is possible to load several drivers.
- The class *DriverManager* manages the loaded driver(s)

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Step 2 : Connecting to a Database (1/2)

- JDBC URL for a database
  - Identifies the database to be connected
  - Consists of three-part:

jdbc:<subprotocol>:<subname>



Ex) // 2. Establish Connection
Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/college", "root", "Mysql@1234");

The syntax for the name of the database is a little messy and is unfortunately vendor specific

Step 2 : Connecting to a Database (2/2)

- The *DriverManager* allows you to connect to a database using the specified JDBC driver, database location, database name, username and password.
- It returns a *Connection* object which can then be used to communicate with the database.

```
// 2. Establish Connection
Connection con = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/college",      "root",  "Mysql@1234"
                );
```

JDBC URL
Vendor of database, Location of database server and name of database

Username

Password

D Y PATIL UNIVERSITY
NAVI MUMBAI

Step 3 : Executing SQL (1/2)

- Statement object
  - Can be obtained from a Connection object

    Statement statement = connection.createStatement();

  - Sends SQL to the database to be executed
- Statement has three methods to execute a SQL statement:
  - **executeQuery()** for QUERY statements
    - Returns a ResultSet which contains the query results
  - **executeUpdate()** for INSERT, UPDATE, DELETE, or DDL statements
    - Returns an integer, the number of affected rows from the SQL
  - **execute()** for either type of statement

Step 3 : Executing SQL (2/2)

- Execute a select statement

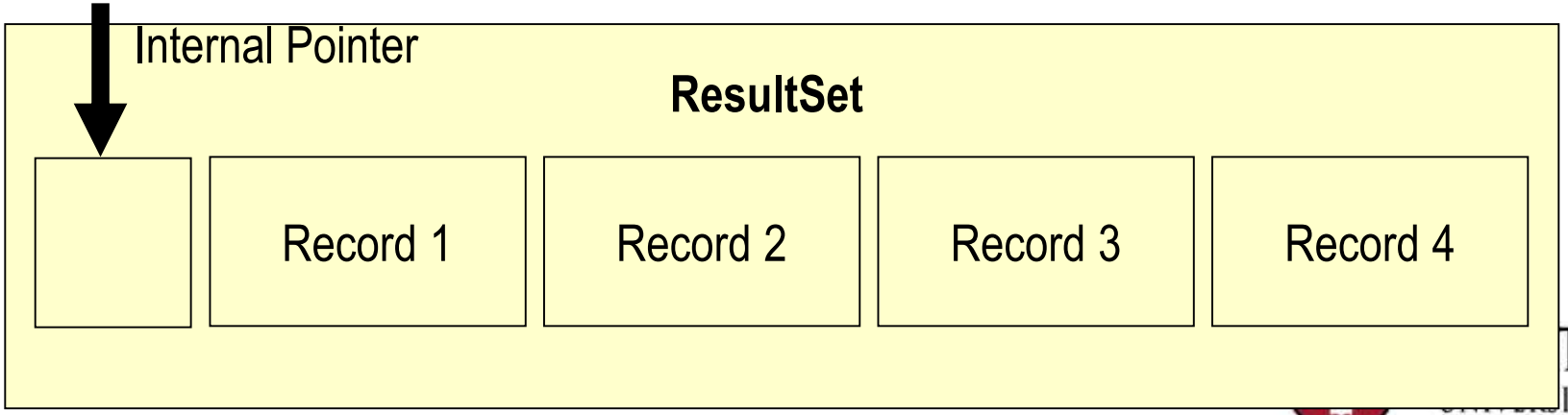
```
Statement stmt = conn.createStatement();
ResultSet rset = stmt.executeQuery
  ("select STUDENT_ID, STATUS from STUDENT");
```

- Execute a delete statement

```
Statement stmt = conn.createStatement();
int rowcount = stmt.executeUpdate
                    ("delete from STUDENT
                      where STUDENT_ID = 1011");
```

Step 4 : Processing the Results (1/2)

- JDBC returns the results of a query in a ResultSet object
  - ResultSet object contains all of the rows which satisfied the conditions in an SQL statement
- A ResultSet object maintains a cursor pointing to its current row of data
  - Use next() to step through the result set row by row
    - next() returns TRUE if there are still remaining records
  - getString(), getInt(), and getXXX() assign each value to a Java variable



The internal pointer starts one before the first record

Step 4 : Processing the Results (2/2)

- Example

```
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT ID, name, score FROM table1");

While (rs.next()){
        int id = rs.getInt("ID");
        String name = rs.getString("name");
        float score = rs.getFloat("score");
        System.out.println("ID=" + id + " " + name + " " + score);}
```

NOTE

You must step the cursor to the first record before read the results
This code will not skip the first record

ID	name	score
1	James	90.5
2	Smith	45.7
3	Donald	80.2

Table1

Output

ID=1 James 90.5
ID=2 Smith 45.7
ID=3 Donald 80.2

D Y PATIL UNIVERSITY
NAVI MUMBAI

Step 5 : Closing Database Connection

- It is a good idea to close the Statement and Connection objects when you have finished with them
- Close the ResultSet object

  rs.close();
- Close the Statement object

  stmt.close();
- Close the connection

  connection.close();

Putting It All Together – A Simple Example

- Let's put all these pieces together and develop a Java application that will connect to our `college` database , execute a query, and return the results.
- This application will show, in the simplest terms, how to load the JDBC driver, establish a connection, create a statement, have the statement executed, and return the results to the application.

Console based JDBC example

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
public class DBTestClass {
                                public static void
main(String[] args) {
                        try {
                           // 1. Load JDBC Driver


Class.forName("com.mysql.cj.jdbc.Driver");
                           // 2. Establish Connection
                           Connection con =
DriverManager.getConnection(

"jdbc:mysql://localhost:3306/college",
                                 "root",
                                 "Mysql@1234"
                           );
```

```
// 3. Create Statement
                           Statement stmt = con.createStatement();

                           // 4. Execute Query
                           ResultSet rs = stmt.executeQuery("SELECT *
FROM student");

                           // 5. Process Result
                           while (rs.next()) {
                              System.out.println(
                                    rs.getInt("id") + "  " +
rs.getString("name")
                              );
                           }

                           // 6. Close Connection
                           rs.close();
                           stmt.close();
                           con.close();
                        } catch (Exceptio
                           e.printStackTra
                        }
                     }
                  }
```

```
1  saloni
2  poonam
3  Arun
4  jai
5  Raju
```

Note Regarding Static Method `forName`

- The database driver must be loaded before connecting to the database. The static method `forName` of class `Class` is used to load the class for the database driver.
- This method throws a checked exception of type `java.lang.ClassNotFoundException` if the class loader cannot locate the driver class.
- To avoid this exception, you need to include the `mysql-connector-j-9.6.0.jar` in your program's classpath when you execute the program.

Connection Interface

The **Connection interface** is part of the java.sql package.
It represents a **session between a Java application and a database**.

A Connection object is used to:

- Establish communication with the database
- Create SQL statements
- Manage transactions

The Connection object is obtained using the **DriverManager** class.

```
Connection con = DriverManager.getConnection(
   "jdbc:mysql://localhost:3306/college",  "root",   "" );
```

Selected Methods In `java.sql.Connection`

Method	Purpose
Statement createStatement()	Returns a statement object that is used to send SQL to the database.
PreparedStatement prepareStatement(String sql)	Returns an object that can be used for sending parameterized SQL statements.
CallableStatement prepareCall(String sql)	Returns an object that can be used for calling stored procedures.
DatabaseMetaData getMetaData()	Gets an object that supplied database configuration information.
boolean isClosed()	Reports whether the database is currently open or not.
void commit()	Makes all changes permanent since previous commit.rollback.
void rollback()	Undoes and discards all changes done since the previous commit/rollback.
void setAutoCommit (boolean yn)	Restores/removes auto-commit mode, which does an automatic commit after each statement. The default case is AutoCommit is on.
void close()	Closes the connection and releases the JDBC resources for the connection.

JDBC Statements

- If the `Connection` object can be viewed as a cable between your application program and the database, an object of Statement can be viewed as a cart that delivers SQL statements for execution by the database and brings the result back to the application program.
- Once a `Connection` object is created, you can create statements for executing SQL statements as follows:

```
Statement statement = connection.createStatement();
```

- JDBC provides three types of statements:
  - Statement
  - PreparedStatement
  - CallableStatement

Creating Statements (cont.)

- The methods illustrated in the previous table are invoked on the Connection object returned by the JDBC driver manager.
- The connection is used to create a Statement object.
- The Statement object contains the methods that allow you to send the SQL statements to the database.
- Statement objects are very simple objects which allow you to send SQL statements as Strings.
- Here is how you would send a select query to the database:

```
Statement myStmt = connection.createStatement();
ResultSet myResult;
myResult = myStmt.executeQuery( "SELECT * FROM bikes;");
```

More on ResultSet later

Creating Statements (cont.)

- The different SQL statements have different return values. Some of them have no return value, some of them return the number of rows affected by the statement, and others return all the data extracted by the query.
- To handle these varied return results, you'll need to invoke a different method depending on what type of SQL statement you are executing.
- The most interesting of these is the SELECT statement that returns an entire result set of data.
- The following table highlights some of the methods in `java.sql.statement` to execute SQL statements.

Some Methods in java.sql.statement to Execute SQL Statements

SQL statement	JDBC statement to use	Return Type	Comment
SELECT	executeQuery(String sql)	ResultSet	The return value will hold the data extracted from the database.
INSERT, UPDATE, DELETE, CREATE, DROP	executeUpdate(String sql)	int	The return value will give the count of the number of rows changed , or zero otherwise.
Stored procedure with multiple results	execute(String sql)	boolean	The return value is true if the first result is a ResultSet, false otherwise.

Exercise Questions

Write a console-based JDBC program to:

- Connect to MySQL database
- Display all records from the student table

Table:

student(id, name, marks)

Write a console-based JDBC program to:

- Insert record into the database

Write a JDBC program to:

- Accept student ID from user
- Display student details if found
- Display message if record does not exist

Add GUI to all programs .

The `PreparedStatement` Interface

- In the previous examples, once we established a connection to a particular database, it was used to send an SQL statement from the application to the database.
- The `Statement` interface is used to execute static SQL statements that contain no parameters.
- The `PreparedStatement` interface, which extends the Statement interface, is used to execute a precompiled SQL statement with or without `IN` parameters.
- Since the SQL statements are precompiled, they are extremely efficient for repeated execution.

The PreparedStatement Object

- A PreparedStatement object holds precompiled SQL statements
- Use this object for statements you want to execute more than once
- A PreparedStatement can contain variables (?) that you supply each time you execute the statement

```
// Create the prepared statement
PreparedStatement pstmt = con.prepareStatement(“
            UPDATE table1 SET status = ? WHERE id =?”)
// Supply values for the variables
pstmt.setString (1, “out”);
pstmt.setInt(2, id);
// Execute the statement
pstmt.executeUpdate();
```

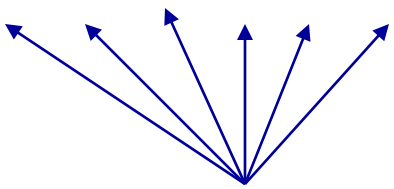
D Y PATIL
UNIVERSITY
NAVI MUMBAI

The `PreparedStatement` Interface

(cont.)

- A `PreparedStatement` object is created using the `prepareStatement` method in the `Connection` interface.

```
PreparedStatement pstmt = connection.prepareStatement
  ("insert into bikes (bikename, size, color,
      cost, purchased, mileage) +
       values ( ?, ?, ?, ?, ?, ?)" );
```



Placeholders for the values that will be dynamically provided by the user.

The `PreparedStatement` Interface

(cont.)

- As a subinterface of `Statement`, the `PreparedStatement` interface inherits all the methods defined in `Statement`. It also provides the methods for setting parameters in the object of `PreparedStatement`.
- These methods are used to set the values for the parameters before executing statements or procedures.
- In general, the set methods have the following signature:

```
setX (int parameterIndex, X value);
```

where `X` is the type of parameter and `parameterIndex` is the index of the parameter in the statement.

The `PreparedStatement` Interface

- As an example, the method (cont.)

```
setString( int parameterIndex, String value)
```

sets a String value to the specified parameter.

- Once the parameters are set, the prepared statement is executed like any other SQL statement where `executeQuery()` is used for SELECT statements and `executeUpdate()` is used for DDL or update commands.
- These two methods are similar to those found in the `Statement` interface except that they have no parameters since the SQL statements are already specified in the `prepareStatement` method when the object of a `PreparedStatement` is created.

```
import java.sql.*;
class InsertPrepared{
public static void main(String args[]){
try{
Class.forName("com.mysql.cj.jdbc.Driver");
Connection con=DriverManager.getConnection(
"jdbc:mysql://localhost:3306/college","root","");
PreparedStatement stmt=con.prepareStatement("insert into Emp
values(?,?)");
stmt.setInt(1,101);//1 specifies the first parameter in the query
stmt.setString(2,"Ratan");
int i=stmt.executeUpdate();
System.out.println(i+" records inserted");
con.close();
}catch(Exception e){ System.out.println(e);}
}}
```

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Find studentUsingPreparedStatement

```
import javax.swing.*;
import java.awt.event.*;
import java.sql.*;

public class StudentSearchPS implements ActionListener {

   JFrame frame;
   JLabel lblId;
   JTextField txtId;
   JButton btnSearch;
   JTextArea taResult;

   Connection con;
   PreparedStatement ps;

  public StudentSearchPS() {

      frame = new JFrame("Student Search Using JDBC");
      frame.setSize(400, 350);
      frame.setLayout(null);

      lblId = new JLabel("Student ID:");
      lblId.setBounds(30, 30, 100, 25);
      frame.add(lblId);
```

```
 txtId = new JTextField();
      txtId.setBounds(130, 30, 150, 25);
      frame.add(txtId);

      btnSearch = new JButton("Search");
      btnSearch.setBounds(130, 70, 100, 30);
      frame.add(btnSearch);

      taResult = new JTextArea();
      taResult.setBounds(30, 120, 320, 150);
      taResult.setEditable(false);
      frame.add(taResult);

      btnSearch.addActionListener(this);

      frame.setVisible(true);

frame.setDefaultCloseOperation(JFrame.EXIT_ON_CL
OSE);

      connectDB();
   }
```

```
// Database connection method
void connectDB() {
   try {
      Class.forName("com.mysql.cj.jdbc.Driver");
      con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college",
            "root",
            "Mysql@123"
      );

      ps = con.prepareStatement(
            "SELECT * FROM student WHERE id = ?"
      );

   } catch (Exception e) {
      taResult.setText("Database connection error");
   }
}
```

```
 // Event handling
   public void actionPerformed(ActionEvent
e) {

      try {
         int id =
Integer.parseInt(txtId.getText());

         ps.setInt(1, id);
         ResultSet rs = ps.executeQuery();

         if (rs.next()) {
            taResult.setText(
                  "ID   : " + rs.getInt("id") + "\n" +
                  "Name : " +
rs.getString("name") + "\n" +
                  "Dept : " + rs.getString("dept")
            );
    } else {
            taResult.setText("No record
found");
         }


      } catch (Exception ex) {
         taResult.setText("Invalid input
or error");
      }
   }

   public static void main(String[]
args) {
      new StudentSearchPS();
   }
```

Output

The CallableStatement Object

- A CallableStatement object holds parameters for calling stored procedures.
- A callable statement can contain variables that you supply each time you execute the call.
- When the stored procedure returns, computed values (if any) are retrieved through the CallababbleStatement object.

How to Create a Callable Statement

- Register the driver and create the database connection.
- Create the callable statement, identifying variables with a question mark (?).

```
CallableStatement cstmt = conn.prepareCall("{call " +
                    ADDITEM + "(?,?,?)}");
```

How to Execute a Callable Statement

1. Set the input parameters.

```
cstmt.setXXX(index, value);
```

2. Execute the statement.

```
cstmt.execute(statement);
```

3. Get the output parameters.

```
var = cstmt.getXXX(index);
```

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Result Sets

- A ResultSet object is similar to a 2D array. Each call to `next()` moves to the next record in the result set. You must call `next()` before you can see the first result record, and it returns false when there are no more result records (this makes it quite convenient for controlling a while loop). (Also remember that the `Iterator next()` returns the next object and not a true/false value.)
- The class ResultSet has "getter" methods `getBlob()`, `getBigDecimal()`, `getDate()`, `getBytes()`, `getInt()`, `getLong()`, `getString()`, `getObject()`, and so on, for all the Java types that represent SQL types for a column name and column number argument.

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Result Sets (cont.)

- A default `ResultSet` object is not updatable and has a cursor that only moves forward.
- Many database drivers support scrollable and updatable `ResultSet` objects.
  - **Scrollable result sets** simply provide a cursor to move backwards and forwards through the records of the result set.
  - **Updatable result sets** provide the user with the ability to modify the contents of the result set and update the database by returning the updated result set.

NOTE: Not all updates can be reflected back into the database. It depends on the complexity of the query and how the data in the result set was derived. In general, base relation attribute values can be modified through an updatable result set.

ResultSet Constants for Specifying Properties

ResultSet static type constant	Description
TYPE_FORWARD_ONLY	Specifies that the ResultSet cursor can move only in the forward direction, from the first row to the last row in the result set.
TYPE_SCROLL_INSENSITIVE	Specifies that the ResultSet cursor can scroll in either direction and that the changes made to the result set during ResultSet processing are not reflected in the ResultSet unless the database is queried again.
TYPE_SCROLL_SENSITIVE	Specifies that the ResultSet cursor can scroll in either direction and that changes made to the result set during ResultSet processing are reflected immediately in the ResultSet.
ResultSet static concurrency constant	**Description**
CONCUR_READ_ONLY	Specifies that the ResultSet cannot be updated (i.e. changes to it will not be reflected into the database).
CONCUR_UPDATABLE	Specifies that the ResultSet can be updated (i.e. changes to it will be reflected into the database using ResultSet update methods).

ResultSet Examples

Syntax for Creating a Scrollable ResultSet

```
Statement stmt = con.createStatement(
        ResultSet.TYPE_SCROLL_INSENSITIVE,
        ResultSet.CONCUR_READ_ONLY
);

ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

Cursor can move forward & backward

Data cannot be modified

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Syntax for Creating an Updatable ResultSet

```
Statement stmt = con.createStatement(
        ResultSet.TYPE_SCROLL_INSENSITIVE,
        ResultSet.CONCUR_UPDATABLE
);

ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

Cursor can move both directions

Data can be updated directly

Common Cursor Methods

```
rs.next();       // Move to next record
rs.previous();   // Move to previous record
rs.first();      // Move to first record
rs.last();       // Move to last record
rs.absolute(3);  // Move to 3rd row
```

Scrollable and updatable ResultSet must be created explicitly using createStatement().

Default ResultSet is forward-only and read-only.